

Linked Lists

Mostafa Faik

September 29, 2024

Introduction

In this assignment, we explored the fundamentals of linked data structures by implementing a simple `linked list` class in Java. Unlike primitive data structures like arrays, linked structures utilize references or pointers to connect different data elements, allowing for greater flexibility and dynamic memory management. This report discusses the implementation of a basic `linked list` in Java, with methods to add, remove, find, and append items, as well as comparisons with arrays, and the design of a stack using `linked lists`.

Linked list

The first task involved creating a `linked list` data structure that can store integers. A `linked list` consists of a sequence of nodes, each containing a value (data) and a reference to the next node. The primary components of the `linked list` are as follows:

- The first node is referred to as the head.
- The last node points to null, marking the end of the list.
- A reference to the head allows access to the entire list.

We implemented the following methods for the `linked list` as well:

- `add(int item)`: Adds a new integer as the first cell in the list.
- `length()`: Returns the number of nodes in the `linked list`.
- `find(int item)`: Checks if a given integer exists in the list, returning true or false.
- `remove(int item)`: Removes the specified integer from the list if it exists.

- `append(LinkedList b)`: Appends another `linked list` to the current list. This operation works by traversing the first list until the last node and setting its next reference to the first node of the second list.

Benchmarks

Varying Size of List A (Append to Fixed List B):

The second task involved setting up benchmarks to analyze the performance of the `append` operation in the `linked list`. We varied the size of the first `linked list` (list A) while appending it to a fixed-size second `linked list` (list B).

The following table summarizes the results of the benchmarks for appending a growing `linked list` (List A) to a fixed-size `linked list` (List B):

Case	n	Time
1	100	40.41 ns
2	200	83.32 ns
3	400	171.14 ns
4	800	335.61 ns
5	1600	672.3 ns
6	3200	1365.06 ns
7	6400	2739.36 ns

Table 1: Append Time for Varying Sizes of List A (Fixed List B)

From the benchmark results, we can observe that as the size of list A increases, the time taken to append it to list B also increases. This behavior aligns with the expected $O(n)$ time complexity for the `append` operation, where n represents the size of list A. The linear relationship between the size of list A and the execution time confirms that the `append` operation depends on traversing list A to its end before connecting it to list B.

Swapping Sizes of Lists A and B:

We then conducted a second benchmark by fixing the size of list A and increasing the size of list B to observe the effect on performance. The results for this benchmark are shown in the following table:

Case	n	Time
1	100	41.04 ns
2	200	43.34 ns
3	400	42.24 ns
4	800	43.26 ns
5	1600	43.50 ns
6	3200	44.73 ns
7	6400	43.42 ns

Table 2: Append Time for Varying Sizes of List B (Fixed List A)

In this scenario, the time taken for appending list A to list B remained fairly consistent. This indicates that the operation’s time complexity primarily depends on the size of list A, not list B. The flat graph shows minimal variance in execution time, further confirming this behavior.

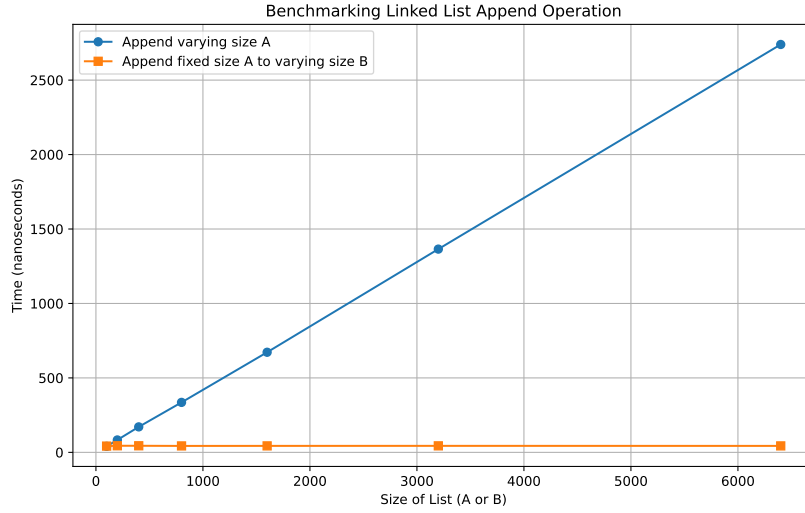


Figure 1: Benchmarking Append Operation

In the graph, we can observe that for list A, the time taken increases linearly as the size of the list grows, demonstrating the $O(n)$ complexity for the append operation. Meanwhile, for list B, the time remains relatively stable because the size of list A is fixed, and the append operation only traverses list A, not list B. This difference highlights the nature of **linked lists**, where traversal is necessary to reach the end before appending.

Comparing Linked Lists to Arrays

In this task, we compared the implementation of the append function using arrays instead of `linked lists`. If we were to use arrays, the implementation would involve allocating a new array of sufficient size to accommodate the combined elements of the two lists. This would typically require the following steps:

- Allocation: Create a new array with a size equal to the sum of the lengths of both lists.
- Copying: Copy the elements from the first array and the second array into the new array.
- Assignment: The new array would then represent the appended structure.

Impact on Benchmarks:

Using arrays introduces potential inefficiencies, especially in scenarios where the array needs to be resized. Specifically, every time an array reaches its capacity, a new, larger array must be allocated, and existing elements copied over an $O(n)$ operation. It requires:

- Reallocation: Allocating a new array and copying existing elements, which is an $O(n)$ operation.
- Memory Overhead: Arrays also have a fixed size, leading to the need for preallocation and possible wasted space.

Implementing a Stack with Linked Lists

In the final task, we explored how to implement a stack data structure using our `linked list` implementation. A stack follows the **Last-In-First-Out** (LIFO) principle, where the last element added is the first to be removed.

Stack Operations:

To implement the stack using the `linked list`, we would use the head of the `linked list` as the top of the stack. The two primary operations would be:

- Push: To add an element, create a new node with the given value and point it to the current head of the `linked list`. Update the head to this new node.
- Pop: To remove the top element, we store the value of the head node, update the head to the next node, and return the stored value.

Pros and Cons of Linked List Stacks:

Pros:

- **Dynamic Size:** The stack can grow and shrink as needed, without the need for resizing or preallocation.
- **No Overflow:** The stack can grow indefinitely until system memory is exhausted, unlike an array that has a fixed size.

Cons:

- **Memory Overhead:** Each node in a **linked list** requires additional memory to store the reference to the next node.
- **Cache Locality:** Since **linked list** nodes are not stored contiguously in memory, accessing elements may be slower compared to arrays, which benefit from better cache performance.

Expected Performance Comparison:

In terms of expected execution time:

- **Array-Based Stack:** The push operation can be $O(1)$ if there is space, but resizing (when the array is full) can make it $O(n)$. Pop is always $O(1)$.
- **Linked List Stack:** Both push and pop operations consistently remain $O(1)$, providing more reliable performance.

Conclusion

This assignment provided a comprehensive understanding of linked data structures, particularly **linked lists**, and their operational characteristics. We implemented a **linked list** class, analyzed its performance through benchmarks, compared it with arrays, and designed a stack using **linked lists**. The findings reinforced the benefits of linked structures, especially in dynamic memory scenarios, while highlighting trade-offs such as memory overhead and access times. Overall, **linked lists** offer significant advantages in flexibility and dynamic sizing, making them a powerful choice for various programming needs. However, arrays can still be preferable when random access and cache locality are priorities.